

Technical Report: Preparing Historical Equity Data for Machine Learning

Course: Data Analytics and Artificial Intelligence

Author: Cesar Juarez

Date: September 2026

Deliverable: Project 2 Technical Report and pdf with code structure

Prepared Dataset File: stock_prepared.csv (2.85 GB)

1. Executive Summary

In this project, we took nearly five decades of historical trading data (1970 to 2018) and transformed it from raw market records into a clean, leakage-free dataset designed specifically to train predictive machine learning models. In our earlier audit, we inspected the numbers from an ‘eagle view” (a general data scouting). Here, we rolled up our sleeves to fix structural bugs, engineer real-world trend indicators, and guarantee our math doesn't accidentally leak future information into the past.

Starting with 20,973,889 price observations across 6,460 security listings, we removed structural file artifacts, computed 7 time-series indicators strictly isolated by security ticker, pruned the initial 20-day warm-up window, and applied one-hot encoding to exchange and sector labels. The finalized dataset contains 20,860,334 production-ready rows across 29 structured features, with zero missing values and zero duplicate records. The final prepared dataset was successfully exported to Google Drive as stock_prepared.csv (2.85 GB).

2. Data Cleaning & Integrity Decisions

2.1 Purging the Injected Header Row

During an upstream concatenation of the master stock listing (df_stocks), a duplicated table header (['Symbol', 'NYSE', ...]) was accidentally injected at index 74. Purging this record eliminated the corrupted entry and left a verified catalog of exactly 6,459 unique, active stock tickers.

2.2 Retaining Non-Operating Assets (Unknown Sectors)

Our initial audit identified 1,440 securities with missing sector and industry classifications. Dropping these records would have introduced severe survivorship and selection bias by deleting investment vehicles that do not belong to traditional corporate operating sectors—such as Exchange-Traded Funds (ETFs), real estate holding trusts, and SPACs. We preserved all 1,440 assets by assigning them to an explicit 'Unknown' sector category, allowing machine learning models to treat them as their own distinct structural asset group.

2.3 Auditing Historical Multi-Million Dollar Closes vs. Floating-Point Glitches

- **Multi-Million Dollar Closes (SCON, CODA, TOPS):** Several micro-cap companies displayed historical closing prices above 1,000,000 USD on trading volumes under 400 shares. These are legitimate historical events resulting from repeated reverse stock splits (such as 1-for-50 consolidations) executed to remain compliant with exchange listing rules. We retained these records because they represent real market history; however, to prevent large nominal share prices from distorting model loss functions, our feature pipeline emphasizes normalized percentage returns rather than raw dollar prices.
- **Vendor Overflow Error (Aaron's Inc. - AAN):** On February 27, 1987, Aaron's Inc. traded normally at 1.30 USD, but its adjusted close column blew up to an impossible 18.9 quintillion USD due to an upstream vendor divide-by-zero bug during a stock split adjustment. We preserved the trading session because the raw trade figures (open, high, low, close, volume) were completely sound, while excluding the corrupted adj_close column entirely from our feature matrix.

3. Time-Series Feature Engineering & Formulations

Raw daily stock prices provide only a static snapshot of trading. To give machine learning models predictive capability, we engineered indicators capturing short-term momentum, medium-term trend baselines, and market volatility. Crucially, every single rolling calculation was executed strictly grouped by ticker (`df.groupby('ticker')`) to ensure that no stock's indicators could ever leak into another company's timeline.

- **1. Previous-Day Closing Price (close_lag_1)**

$$close_lag_1 = Close_{t-1}$$

Real-World Context: Captures yesterday's closing price as a direct numerical anchor for sequential time-series modeling.

- **2. Daily Dollar Price Change (price_change)**

$$price_change = Close_t - Close_{t-1}$$

Real-World Context: Measures the net dollar shift between consecutive trading sessions.

- **3. Daily Percentage Return (daily_return)**

$$daily_return = \frac{Close_t - Close_{t-1}}{Close_{t-1}}$$

Real-World Context: Normalizes performance across price scales. A 1.00 USD gain on a 5.00 USD stock (+20%) is radically different from a 1.00 USD gain on a 500.00 USD stock (+0.2%). This puts all 6,459 assets on a fair, comparable scale.

- **4. 5-Day Moving Average (ma_5)**

$$ma_5 = \frac{1}{5} \sum_{i=0}^4 Close_{t-i}$$

Real-World Context: Filters out day-to-day noise to reveal short-term, 1-week price momentum.

- **5. 20-Day Moving Average (ma_20)**

$$ma_20 = \frac{1}{20} \sum_{i=0}^{19} Close_{t-i}$$

Real-World Context: Establishes a baseline 1-month trend line. Prices trading consistently above this line reflect upward momentum.

- **6. 20-Day Rolling Volatility (volatility_20)**

$$volatility_20 = \sqrt{\frac{1}{19} \sum_{i=0}^{19} (R_{t-i} - \bar{R})^2}$$

Real-World Context: Measures return dispersion over the past month, signaling whether a security is trading calmly or undergoing a high-risk regime shift.

- **7. Intraday Range Spread Percentage (daily_range_pct)**

$$daily_range_pct = \frac{High_t - Low_t}{Low_t}$$

Real-World Context: Captures intraday price swings relative to share price. A stock may close with 0% net daily change while experiencing a 10% intraday swing between buyers and sellers.

4. Pipeline Architecture & Structural Transformations

To handle 20.9 million rows without exceeding memory constraints, we structured the workflow into four distinct stages: Raw Ingestion, Grouped Feature Engineering, Memory-Optimized Merging & Encoding, and Final Integrity Auditing.

[Note: Insert the rendered pipeline workflow diagram generated in your Colab notebook right here]

4.1 Missing Value Treatment: Pruning the 20-Day Warm-Up Horizon

Rolling aggregations require historical lookback periods. For any given stock, Day 1 has no previous trading day, and Days 1 through 19 lack the 20 trading days required to compute a 20-day moving average. We avoided zero-imputation because assigning 0.0 falsely implies zero price movement, a 0% return, and completely flat volatility. Instead, initial warm-up rows were pruned across all stocks. This removed 113,555 records (<0.55% of the data), ensuring that every single row available for machine learning contains a complete, 20-day historical feature set.

4.2 Categorical One-Hot Encoding

Machine learning algorithms interpret numbers quantitatively. Arbitrarily assigning NASDAQ = 1 and NYSE = 2 imposes a false numerical hierarchy, implying that NYSE is 'greater than' NASDAQ. To prevent this, we used one-hot encoding, converting exchanges and economic sectors into binary flags (1 for yes, 0 for no). To conserve RAM, binary columns were stored as int8 and price decimals as float32, keeping peak Colab memory at 1.65 GB.

5. Final Quality Audit & Dataset Structure

A pre-export audit verified that the prepared dataset meets production quality standards:

Quality Check Metric	Standard Expected	Measured Output	Final Status
Total Completed Rows	Over 20 Million Rows	20,860,334 Rows	PASSED
Total Structured Features	Full Feature Set	29 Columns	PASSED
Missing / Blank Values (NaN)	Zero Missing Values	0 Nulls Remaining	PASSED
Duplicate Entity-Date Days	Zero Duplicate Days	0 Duplicates Found	PASSED
Historical Timeline	Full Historical Range	Jan 02, 1970 to Aug 24, 2018	PASSED
Export File Size on Disk	Persisted in Google Drive	2.85 GB	PASSED

6. Recommendations for Project 3 Modeling

1. Enforce Chronological Splits: Because financial markets evolve chronologically, standard random cross-validation allows future information to leak into past predictions. The dataset should be partitioned chronologically (e.g., training on 1970–2012, validating on 2013–2015, and testing on 2016–2018).

2. Target Relative Returns, Not Raw Dollar Prices: Predicting raw closing prices across a universe spanning penny stocks and multi-thousand-dollar equities introduces scale distortion. Models should target future percentage returns or binary trend breakout events (e.g., whether tomorrow's close crosses above `ma_20`).

3. Incorporate Macroeconomic Signals: Subsequent model iterations will benefit from integrating benchmark interest rates, Treasury yield curves, and market index returns to account for broader macroeconomic conditions.